arXiv:2508.20134v1 [cs.AI] 26 Aug 2025

QAgent: An LLM-based Multi-Agent System for Autonomous OpenQASM programming

Zhenxiao Fu, Fan Chen, Lei Jiang

Indiana University Bloomington.

Contributing authors: zhfu@iu.edu; fc7@iu.edu; jiang60@iu.edu;

Abstract

Noisy Intermediate-Scale Quantum (NISQ) devices have begun to exhibit early quantum advantages on classically intractable problems, spanning physics simulations to Gaussian boson sampling. Yet, realizing these benefits remains challenging for non-experts, primarily due to the complexities of programming in Open Quantum Assembly Language (OpenQASM). Although Large Language Model (LLM)-based agents have shown promise in automating classical programming workflows, their quantum counterparts have largely been restricted to specialized tasks such as quantum chemistry or error correction. In this paper, we present QAgent, an LLM-powered multi-agent system that fully automates OpenQASM programming. By integrating task planning, in-context few-shot learning, retrieval-augmented generation (RAG) for long-term context, predefined generation tools, and chain-of-thought (CoT) reasoning, the agents systematically improve both compilation and functional correctness. Our evaluations demonstrate substantial improvements: across multiple LLMs of varying sizes, QAgent enhances the accuracy of QASM code generation by 71.6% compared to previous static LLM-based approaches. We envision this multi-agent system as a key enabler for democratizing quantum programming, bridging expertise gaps, and accelerating the practical adoption of quantum computing.

Keywords: OpenQASM, LLM, Agent

1 Introduction

Despite inherent noise and limited qubit counts, Noisy Intermediate-Scale Quantum (NISQ) devices have already demonstrated quantum advantages on classically intractable tasks, including physics simulation [1], principal component analysis [2], cryptography [3], chemistry [4], combinatorial optimization [5], and Gaussian boson sampling [6].

To realize these capabilities, Open Quantum Assembly Language (OpenQASM) [7] serves as a critical interface between quantum software development kits (SDKs), such as Qiskit [8, 9] and PennyLane [10], and heterogeneous NISQ hardware platforms, including superconducting [11] and trapped-ion [12] systems. OpenQASM underpins platform-agnostic and platform-specific optimization, mapping, scheduling, evaluation, profiling, and simulation. Although quantum SDKs can automatically produce OpenQASM from high-level circuits, manual OpenQASM programming remains essential. Compiler developers embed high-level information into OpenQASM to guide

optimization [13], experimentalists adjust timing and pulse-level instructions by OpenQASM for fine-tuning [14], and hardware engineers write OpenQASM to address controller and waveform constraints [15].

However, OpenQASM programming poses significant challenges for non-experts due to its quantum-specific abstractions. Its syntax, centered on qubits, gates, pulses, and hybrid control, departs markedly from classical languages like C and Python, demanding fluency across algorithms, calibrations, and hardware constraints [7]. Variations in gate sets, connectivity, and real-time control introduce further hardware dependencies [16]. Additionally, OpenQASM's dynamic control and classical feedback require expertise in concurrency and control systems [7]. Debugging is particularly demanding, as many errors only surface at runtime and necessitate deep knowledge of both quantum algorithms and low-level hardware behavior [17].

Large Language Model (LLM)-powered agents [18, 19] have emerged as promising tools for automating software development, testing, and debugging across languages such as C and Python. For example, GitHub Copilot, integrated with Microsoft VSCode, assists in code management, refactoring, and optimization [20]. While static LLMs have been applied to quantum tasks—including many-body physics [21], quantum state simulation [22], text-to-circuit generation [23–27], circuit synthesis [28], and algorithm explanation [29]—few LLM-powered agents are explicitly designed for quantum computing. Some target quantum chemistry tasks [30] executed on classical hardware, others direct physical labs [31] related to quantum experiments, or assist in designing quantum error correction codes [32]. Additional agents generate high-level SDK code in Qiskit [8, 9] and PennyLane [10], but remain confined to Python abstractions. *To the best of our knowledge, no existing LLM-powered agent system supports OpenQASM programming*, limiting accessibility for non-experts and constraining broader adoption of quantum computing in the NISQ era.

To automate OpenQASM programming on NISQ platforms, we introduce QAgent, a hybrid multi-agent system powered by large language models (LLMs) that seamlessly integrates planning, reasoning, memory retrieval, and tool usage—eliminating the need for human intervention. Unlike traditional workflows that require deep expertise in quantum concepts such as pulses, gates, and qubit-level abstractions, QAgent lowers the barrier to entry by fully managing these complexities through coordinated agent collaboration.

Given a quantum problem, the system decomposes it into sub-tasks. For sub-tasks with clear precedents, it employs in-context few-shot learning to synthesize the corresponding OpenQASM code through an Dynamic-few-shot multi-agent Coder. For complex quantum subroutines that benefit from specialized tools, the system utilizes a Tools-augmented multi-agent Coder that leverages a predefined toolset to produce accurate circuits.

Across both modules, QAgent applies retrieval-augmented generation (RAG), few-shot in-context learning, chain-of-thought (CoT) reasoning and reflection mechanisms to iteratively generate, analyze, debug and refine the code. By combining all the techniques, QAgent substantially improves both syntax and functional pass rates—making quantum programming on NISQ devices more accessible and robust.

2 QAgent Design

QAgent is a hierarchical multi-agent framework designed for autonomous quantum algorithm synthesis in OpenQASM. It consists of two specialized multi-agent Coder modules: the **Dynamic-few-shot Coder** and the **Tools-augmented Coder**, each responsible for tackling different categories of quantum tasks. QAgent supports a hybrid strategy that utilizes both two Coders to cover a variety of task complexity. Each Coder internally orchestrates a series of collaborative LLM-powered agents responsible for generation, validation, and refinement through multi-round self-reflection. This section details the overall architecture and design of QAgent, as well as the internal mechanisms of the two Coders and their hybrid coordination. Importantly, the framework is designed with a pluggable **base LLM**, meaning the underlying model can be replaced on demand depending on task requirements, computational

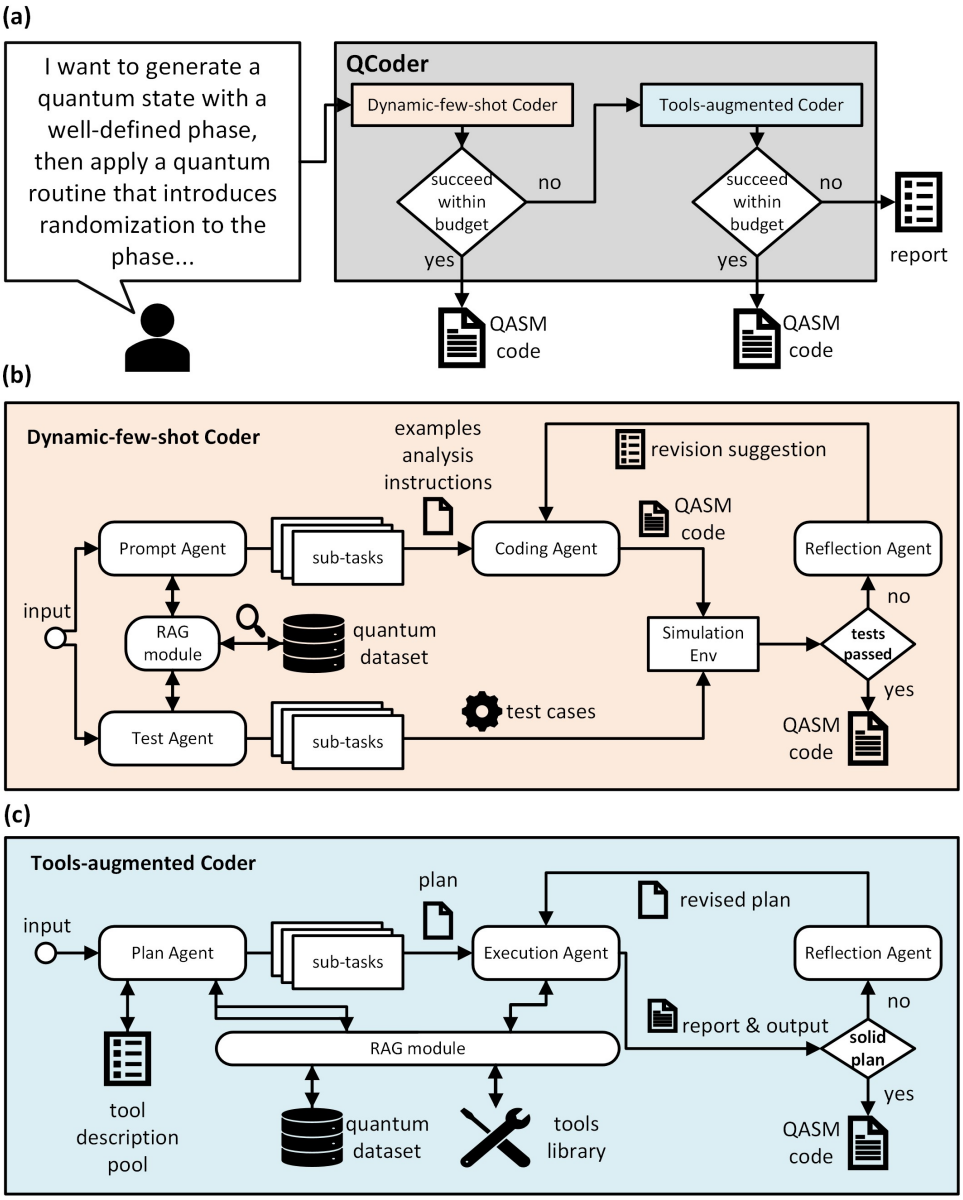


Fig. 1: (a) the overall workflow for QAgent. (b) the detailed design of Dynamic-few-shot Coder. (c) the detailed design of Tool-augmented Coder.

resources, or advancements in language model capabilities. This flexibility ensures QAgent remains adaptable and effective as the foundation models evolve.

2.1 Overall Workflow

As shown in Figure 1 (a), the QAgent workflow begins with a user-provided natural language description of a quantum circuit. Based on this input, QAgent first dispatches the task to the Dynamic-few-shot Coder, which excels at short and structurally regular quantum tasks. If this Coder fails to yield a valid result within limited computational budget (defined as a fixed number of candidate generations and reflection rounds), the task is escalated to the Tools-augmented Coder, which is more capable of handling complex, parameterized quantum tasks.

Each Coder operates as a self-contained multi-agent pipeline which generally involves: (1) problem understanding and decomposition, (2) code generation, (3) test

generation and validation, and (4) iterative reflection and revision. The final output is a syntactically and semantically valid OpenQASM program.

2.2 Dynamic-few-shot Multi-agent Coder

The Dynamic-few-shot Coder (Figure 1 (b)) solves quantum programming tasks by mimicking patterns from known code examples. It is most effective when the target algorithm is well-structured, short, and resembles known canonical quantum algorithms.

Upon receiving the task description, the **Prompt Agent** analyzes the input using both its internal language model and retrieval-augmented generation (RAG). It identifies the most relevant algorithm type and retrieves a set of similar example QASM code from a quantum algorithm database. These retrieved examples are parsed and interpreted to produce a composite in-context learning prompt, which includes: (1) the selected examples, (2) an analysis or summary of the examples, and (3) specific instructions for generating new code in OpenQASM. This prompt is then passed to the **Coding Agent**, which produces several **candidate** QASM programs.

Simultaneously, a **Test Agent** generates a test suite tailored to the task. This agent also leverages RAG to find appropriate testing templates and injects randomized quantum oracles where applicable (to account for the oracle-dependence of many quantum algorithms). Each candidate QASM program is executed in the test environment to check for both syntactic validity and semantic correctness.

If a candidate passes the test, it is returned as the final output. Otherwise, the task is forwarded to the **Reflection Agent**, which observes the full history of the reasoning process—including the examples, prompt construction, code candidates, test results, and any prior edits. Based on this information (serves a CoT prompt), it generates a revised strategy, which is handed back to the Coding Agent to produce new code. This process is repeated over multiple **reflection rounds**, each of which refines the output based on accumulated feedback. By default, the Dynamic-few-shot Coder performs one candidate generation and up to three reflection rounds. Detailed example outputs for each agent can be found in Supplemental Material 7.1.

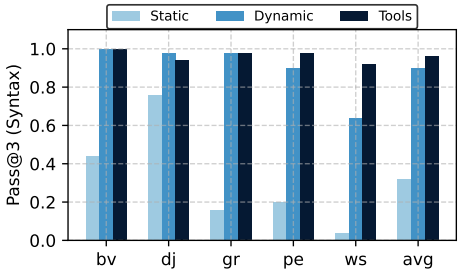
2.3 Tools-augmented Multi-agent Coder

The Tools-augmented Coder (Figure 1 (c)) is designed to handle more complex, less templated quantum tasks that may involve parameter tuning or complex quantum-gate definitions. This Coder operates not by mimicking examples, but by actively planning how to solve the problem using a set of pre-defined, composable quantum programming tools.

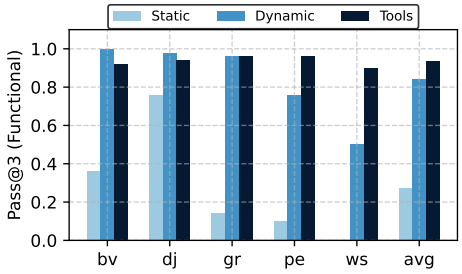
Upon activation, the **Plan Agent** first analyzes the natural language description and determines the type of quantum algorithm required, either based on internal model knowledge or via RAG. It then selects a subset of functional tools from a predefined toolset—such as operations for gate application, oracle generation or helper utilities for formatting and post-processing.

Based on the selected tools and the task specification, the **Plan Agent** constructs detailed plans (**candidates**), consisting of a sequence of actions expressed in natural language and Python code. This plan is then submitted to the **Execution Agent**, which carries out the instructions, invokes the selected tools, and generates OpenQASM code.

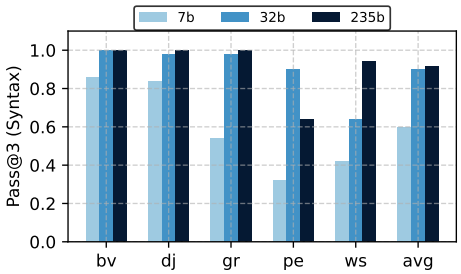
After execution, the output is validated for correctness. If the resulting QASM code meets the desired criteria, it is returned to the user. Otherwise, the entire plan—including the original instruction, the tools used, generated code, test logs, and revision history—is passed to the **Reflection Agent**. The Reflection Agent revises the plan or suggests changes in tool composition or step ordering. As with the Dynamic-few-shot Coder, this revision process is iterated over multiple **reflection rounds**, with the same default setting of one candidate and three rounds. Detailed example outputs for each agent can be found in Supplemental Material 7.2.



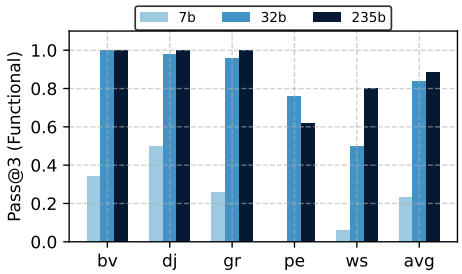
(a) Pass@3 (Syntax) comparison across Static, Dynamic and Tools Coders on Qwen-32B.



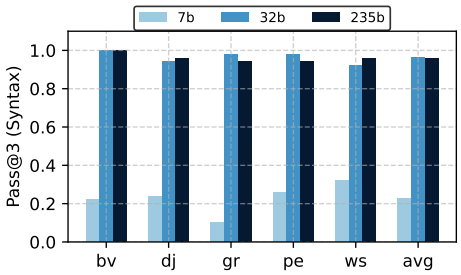
(b) Pass@3 (Functional) comparison across Static, Dynamic and Tools Coders on Qwen-32B.



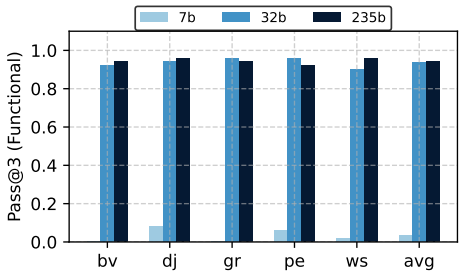
(c) Pass@3 (Syntax) comparison across model sizes using Dynamic Coder.



(d) Pass@3 (Functional) comparison across model sizes using Dynamic Coder.



(e) Pass@3 (Syntax) comparison across model sizes using Tools Coder.



(f) Pass@3 (Functional) comparison across model sizes using Tools Coder.

Fig. 2: Comparison of syntax-level and functional-level pass@3 success rate on different quantum algorithms across Coders (Static, Dynamic, Tools) and models of different sizes (Qwen-7B, 32B, 235B).

2.4 Hybrid Strategy

The hybrid execution mode of QAgent leverages the complementary strengths of the two Coders. The Dynamic-few-shot Coder requires relatively low reasoning capability from the LLM, as it relies primarily on pattern imitation from examples. It performs exceptionally well on well-structured and short tasks. The Tools-augmented Coder, while more dependent on the LLM's analytical and planning capabilities, is better suited for complex tasks requiring parameterization or multi-level quantum-gate composition.

By default, QAgent first applies the Dynamic-few-shot Coder. If it fails to produce a valid output within the allowed budget (3 candidate, 5 reflection rounds), the system automatically switches to the Tools-augmented Coder as a fallback strategy. In future versions, more dynamic routing strategies could be developed—e.g., using task complexity estimation or early failure prediction to make proactive Coder assignments.

This fallback strategy forms the basis of QAgent's robustness and versatility across a wide range of quantum programming tasks, as demonstrated in our experiments in next section.

3 Results

We now demonstrate QAgent's capabilities on single- and multi-algorithm quantum programming tasks, which directly correspond to the level-n setting. Specifically, level-n denotes problem instances where the user query involves n distinct quantum algorithms. Level-1 tasks (single-algorithm) evaluate QAgent's in-context few-shot learning and RAG-based tool invocation capabilities, while level-2 and higher tasks (multi-algorithm) showcase QAgent's ability to perform problem decomposition and systematic task planning. The details of these tasks can be found in Section 5

3.1 Evaluation on Level-1 Problems

We first evaluated QAgent on OpenQASM code generation tasks consisting of a single quantum algorithm problem. Five algorithms were selected — Bernstein-Vazirani (bv), Deutsch-Jozsa (dj), Grover (gr), Phase-Estimation (pe), and W-state Preparation (ws) — for constructing the Level-1 problems benchmark. Three approaches were compared: static few-shot prompting (*Static*) [24], QAgent's Dynamic-few-shot Coder (*Dynamic*), and QAgent's Tools-augmented Coder (*Tools*), as shown in Figure 2. The *Static* baseline reproduces prior work, where a small number (1–5) of example QASM programs are directly appended to the user task description for circuit generation, without any structured chain-of-thought prompt, code analysis, or self-reflection for error correction. The *Dynamic* scheme uses only QAgent's Dynamic-few-shot Coder and the *Tools* scheme uses only the Tools-augmented Coder.

We measured the syntactic and functional success rates (`pass@3`) achieved by Qwen2.5-Coder-32B-Instruct (Qwen-32B) as the base LLM, as illustrated in Figure 2a and Figure 2b. The metric `pass@k` quantifies the proportion of syntactically or functionally correct codes among k generation trials (candidates); further details on the benchmark, approaches, and metrics are provided in the Methods section 5. All three approaches consistently achieved higher syntactic `pass@3` rates compared to functional `pass@3` rates, highlighting the relative ease of generating syntactically valid code.

Among the evaluated approaches, the *Static* method demonstrated the lowest syntactic and functional `pass@3` rates, primarily due to ambiguity in qubit specification and the absence of dynamic reflection rounds. For simpler tasks (bv, dj, gr), QAgent's *Dynamic* approach yielded the highest syntactic and functional success rates, as shorter, structured OpenQASM codes are effectively generated through imitation from few-shot examples. Conversely, the *Tool* approach occasionally invoked incorrect tools or used wrong parameters. However, for more complex problems (pe, ws) requiring parameter tuning and multi-step gate construction, QAgent's *Tool* approach outperformed others. This result demonstrates that parameter extraction and tool invocation are more reliable for longer, unstructured code generation tasks, where few-shot imitation becomes challenging.

3.2 Ablation Studies

3.2.1 Influence of LLM Scale

We analyze QAgent's OpenQASM generation quality with three base LLM back-ends of decreasing size: Qwen3-235B-A22B-Instruct-2507 (235b or Qwen-235B), Qwen2.5-Coder-32B-Instruct (32b or Qwen-32B), and Qwen2.5-Coder-7B-Instruct (7b or Qwen-7B).

Dynamic-few-shot Coder. As Figure 2c and 2d illustrates, Qwen-7B consistently records the lowest syntactic and functional `pass@3` scores across all level-1 tasks. Qwen-32B and Qwen-235B perform similarly well on the simpler tasks (bv, dj, gr), but both models deteriorate on the more complex tasks (pe, ws), with Qwen-235B maintaining a moderate advantage.

Tools-augmented Coder. The same relative ordering emerges: Qwen-7B again fails owing to limited capacity for parameter extraction and tool selection, whereas Qwen-32B and 235B achieve markedly higher `pass@3` rates. Notably, for the complex

pe and ws problems, Qwen-32B and 235B maintain substantially higher `pass@3` values than *Dynamic* scheme (Figure 2e and 2f).

Overall, Qwen-235B achieves the highest `pass@3` rates across both coder workflows. Qwen-32B remains competitive by selecting its strategy: for simple tasks (bv, dj, gr), Dynamic-few-shot coder outperforms the Tools-augmented coder, as imitation of brief examples is more effective than constructing a tool-based plan.

3.2.2 Effect of Reflection Rounds

Using Qwen-32B as base LLM, we measure the average functional `pass@1` rate as a function of CoT-based reflection rounds (Figure 3a and 3b). Here, we fix the default candidate number to 1, thereby enabling a clearer evaluation of the performance contributed by the number of reflection rounds. With *Dynamic* Coder, additional reflections yielded negligible improvement, suggesting that iterative reflection cannot remedy errors arising from example imitation. In contrast, for *Tools*, increasing the number of reflection rounds significantly enhanced code quality, indicating that iterative reflection effectively refines parameter extraction and tool calls.

3.2.3 Impact of Candidate Number

We evaluate the average functional `pass@k` rate as a function of the number of generated candidates k (Figure 3c and Figure 3d) while using Qwen-32B as base LLM. We fix the default reflection rounds to 3, ensuring that the reflection mechanism remains functional and the effect of the candidate number can be highlighted. In both *Dynamic* and *Tools* workflows, increasing k markedly improves success rates, as the probability of at least one valid candidate rises. This outcome underscores the benefit of trial diversity in error-prone code generation tasks.

3.2.4 Contribution of Different Components

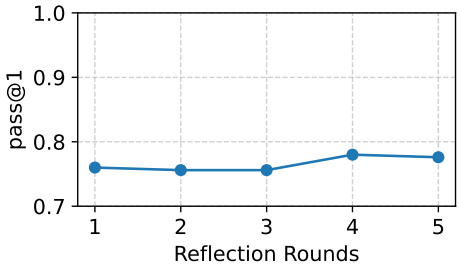
Finally, Figure 3e presents an ablation study of the Dynamic Coder (1 candidate and 3 reflection rounds), revealing that every component, including analysis, CoT-reflection (error correcting suggestions), few-shot in-context learning prompt, contributes meaningfully to success. Removing any element leads to a noticeable performance drop. The largest performance drop occurred when the few-shot prompt was removed (0-shot), resulting in a decrease of up to 69%, whereas the smallest impact came from the analysis component in the prompt, whose removal led to a 5.2% loss. We did not perform an ablation study on the *Tools* Coder, since its process — from plan formulation to tool execution — is tightly coupled and cannot be meaningfully segmented.

3.3 Multi-algorithm Problems

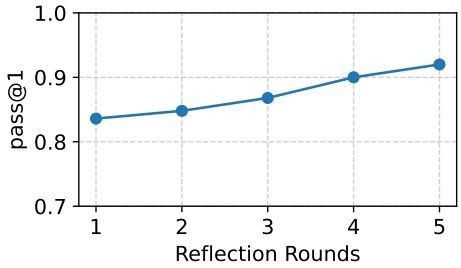
Figure 4 evaluates the full **QAgent** system, which integrates *Dynamic* and *Tools*, across both single-task and multi-algorithm scenarios.

Figure 4a compares performance on the same set of single-task benchmarks used in earlier experiments, with 'p-best' bars representing the best performance of *Dynamic* or *Tools* alone in previous experiments. This direct comparison illustrates the benefit of QAgent's hybrid design: by leveraging *Dynamic* for straightforward problems and *Tools* for complex reasoning, QAgent achieves the highest overall accuracy across all LLM base models. This represents a substantial improvement over using *Dynamic* or *Tools* alone on the same tasks, confirming that the hybrid structure effectively combines the strengths of both paradigms.

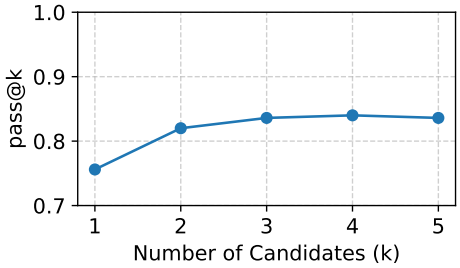
Figure 4b– 4d focus on more challenging scenarios involving combinations of 2 to 4 different quantum algorithms. These multi-algorithm tasks are intrinsically complex because they require QAgent to decompose the overall problem into subtasks first. The results show a clear trend: as the number of involved algorithms increases, accuracy drops substantially. This decline is not only due to the challenge of generating correct QASM code, but also because QAgent must reliably identify and segment subtasks from the user's instructions — a process that becomes increasingly error-prone with task complexity. When handling tasks involving two algorithms, Qwen-235B and 32B



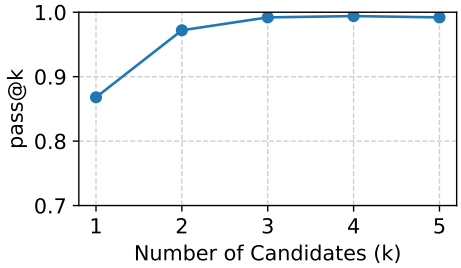
(a) Pass@1 (functional) of Dynamic Coder under different numbers of reflection rounds (single candidate).



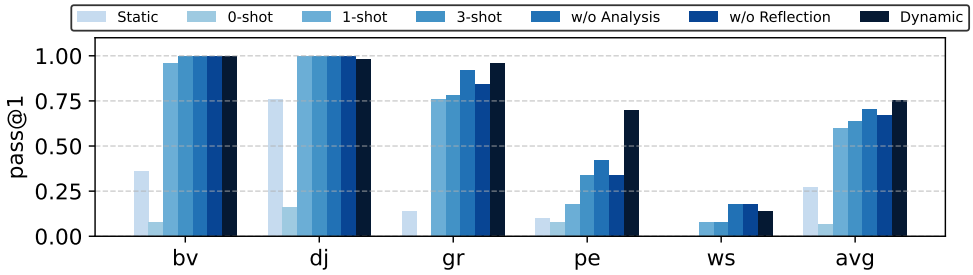
(b) Pass@1 (functional) of Tools Coder under different numbers of reflection rounds (single candidate).



(c) Pass@k (functional) of Dynamic Coder under different numbers of candidates (fixed 3 reflection rounds).



(d) Pass@k (functional) of Tools Coder under different numbers of candidates (fixed 3 reflection rounds).



(e) Pass@1 (functional) ablation study on Dynamic Coder. The comparison includes the static method and various Dynamic Coder settings: different few-shot numbers, removal of analysis, removal of reflection, and the full Dynamic configuration.

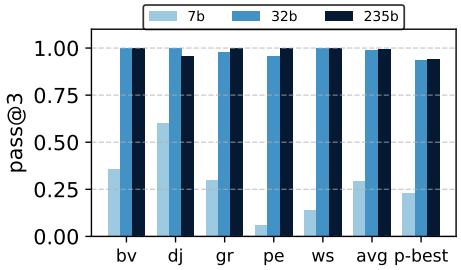
Fig. 3: Analysis of candidate/reflection number and ablation studies on Dynamic Coder (base LLM using Qwen-32B).

can still maintain relatively strong performance (above 50%), but in scenarios involving three or more algorithms, all models begin to struggle, with Qwen-7B failing to produce any successful outputs.

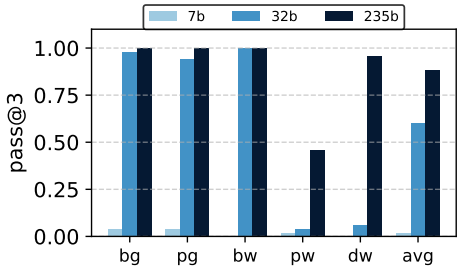
This observation suggests an important direction for future work: improving the ability of LLM-based systems to more accurately decompose user requirements into well-defined subtasks, thereby enabling more robust performance on highly composite tasks.

4 Conclusion

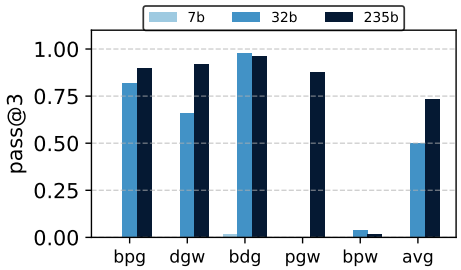
QAgent addresses the complexities of quantum programming on NISQ machines by integrating LLM capabilities with automated problem decomposition, circuit generation, synthesis, debugging, and optimization. By leveraging RAG-based long-term memory, few-shot in-context learning, and COT-based reflection process, QAgent mitigates the need for extensive quantum expertise, enabling non-experts to develop, test, and optimize quantum circuits effectively. Experimental results demonstrate that



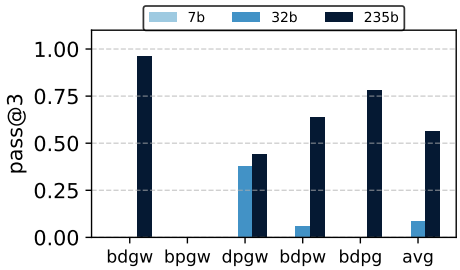
(a) Pass@3 (functional) of QAgent on level-1 problems.



(b) Pass@3 (functional) of QAgent on level-2 problems.



(c) Pass@3 (functional) of QAgent on level-3 problems.



(d) Pass@3 (functional) of QAgent on level-4 problems.

Fig. 4: Pass@3 (functional) results of QAgent across problems with different levels of complexity and different LLM sizes. Here, level-n denotes problems where the user query simultaneously involves n different quantum algorithm.

QAgent achieves up to 71.6% improvements in QASM code generation correctness over prior static LLM text-to-circuit generators and proved effective across different LLM base models. Thus, QAgent substantially lowers the barrier to quantum programming, facilitating broader adoption of quantum computing in the NISQ era.

5 Methods

5.1 Dataset

Level-1 foundamental quantum problems We evaluated QAgent using the QCircuitNet dataset [24], which includes fundamental quantum problems described by natural language prompts enriched with LaTeX-based mathematical notation. Specifically, we selected the following problems:

- *Bernstein-Vazirani (bv)*: Identifies a hidden binary string encoded in a linear Boolean function using a single oracle query.
- *Deutsch-Jozsa (dj)*: Determines whether a black-box Boolean function is constant or balanced via a single quantum evaluation.
- *Grover (gr)*: Performs quadratically accelerated search on unsorted databases using amplitude amplification.
- *Phase Estimation (pe)*: Computes the eigenphase of a unitary operator corresponding to a given eigenstate, foundational for various quantum algorithms.
- *W-state (ws)*: Generates multipartite entangled quantum states known for robust entanglement, beneficial in quantum information processing.

Each problem includes a dedicated Python-based OpenQASM code generator accommodating varying qubit counts and associated test cases for assessing syntactic and functional correctness. These resources adequately support both in-context few-shot learning and RAG-based tool invocation. We set the maximum qubit number to 12 across all kernels to ensure practical simulation feasibility.

Higher-level Multi-algorithm Problems. To assess QAgent's performance on more complex tasks, we constructed composite problems integrating multiple fundamental quantum problems. For instance, in task *bg*, the user specifies an input question that involves both the Bernstein–Vazirani (bv) algorithm and Grover's algorithm (gr). The composite multi-quantum-algorithm Tasks dataset spans tasks that combine two, three, or four algorithms. The following table enumerates all multi-algorithm tasks and the algorithms logically required to solve them:

- **bg**: *Bernstein-Vazirani*, *Grover*
- **pg**: *Phase Estimation*, *Grover*
- **bw**: *Bernstein-Vazirani*, *W-state*
- **pw**: *Phase Estimation*, *W-state*
- **dw**: *Deutsch-Jozsa*, *W-state*
- **bpg**: *Bernstein-Vazirani*, *Phase Estimation*, *Grover*
- **dgw**: *Deutsch-Jozsa*, *Grover*, *W-state*
- **bdg**: *Bernstein-Vazirani*, *Deutsch-Jozsa*, *Grover*
- **pgw**: *Phase Estimation*, *Grover*, *W-state*
- **bpw**: *Bernstein-Vazirani*, *Phase Estimation*, *W-state*
- **bdgw**: *Bernstein-Vazirani*, *Deutsch-Jozsa*, *Grover*, *W-state*
- **bpgw**: *Bernstein-Vazirani*, *Phase Estimation*, *Grover*, *W-state*
- **dpgw**: *Deutsch-Jozsa*, *Phase Estimation*, *Grover*, *W-state*
- **bdpw**: *Bernstein-Vazirani*, *Deutsch-Jozsa*, *Phase Estimation*, *W-state*
- **bdpg**: *Bernstein-Vazirani*, *Deutsch-Jozsa*, *Phase Estimation*, *Grover*

This dataset provides a controlled yet challenging environment for assessing whether a language model–driven agent can infer and correctly decompose complex composite requests into the necessary quantum primitives. By covering combinations from two up to four algorithms, the dataset systematically increases reasoning complexity while maintaining interpretable ground truth for evaluation. More detailed descriptions for these tasks can be found in the GitHub repository.

5.2 LLM Models

To implement QAgent and compare against static LLM baselines, we evaluated several code-capable LLMs: Qwen3-235B-A22B-Instruct-2507 (235b or Qwen-235B), Qwen2.5-Coder-32B-Instruct (32b or Qwen-32B), and Qwen2.5-Coder-7B-Instruct (7b or Qwen-7B). Generation parameters were fixed across all models: temperature = 1.0 and top-p = 1.0. To reduce variance due to stochastic generation, each experiment was repeated five times, and results were averaged.

5.3 Implementation

All LLMs were accessed via cloud APIs, with requests issued from a local machine equipped with an Intel core i5-12600K CPU, 64GB DRAM, and an NVIDIA GeForce RTX3090 GPU. The generated OpenQASM code was simulated and evaluated locally using Qiskit and benchmark-specific test cases.

5.4 Metrics

We evaluated QAgent using the `pass@k` rate [33]. The `pass@k` metric quantifies correctness, where `pass@k` (`Syntax`) measures the proportion of compilable (syntactically valid) code, and `pass@k` (`Functional`) evaluates the proportion of compilable code that passes all functional test cases:

$$\text{pass@}k = 1 - \frac{\binom{n_t - n}{k}}{\binom{n_t}{k}} \tag{1}$$

Here, n_t is the total number of generated samples, k is the number of generation trials, and n is the number of successful (passing) samples.

Schemes. Our experiments evaluate 4 distinct schemes. All methods utilize pretrained LLMs without any additional fine-tuning or LoRA adaptation.

1. *Static Few-Shot* (*Static*) : This baseline reproduces the setup from prior work [24], where a small number (1–5) of example QASM code are appended to the user-provided task description to generate required code. It does not include any structured CoT prompt, code analysis or self-reflection mechanisms for error correction.
2. *Dynamic-few-shot Coder* (*Dynamic*): This scheme involves Dynamic-few-shot Coder alone. The default inference budget is limited as 3 candidate and 5 reflection rounds.
3. *Tools-augmented Coder*(*Tools*) : This scheme involves Tools-augmented Coder alone. The default inference budget is limited as 3 candidate and 5 reflection rounds.
4. *QAgent* : This full system combines both Dynamic-few-shot and Tools-augmented Coders in a hierarchical fallback architecture, as described previously. If the Dynamic-few-shot Coder fails to produce a valid result within a fixed number of candidates and reflection rounds, the task is automatically escalated to the Tools-augmented Coder for further processing.

6 Code and Data Availability

The implementation of the proposed QAgent system will be made publicly available at `https://github.com/fuzhenxiao/QCoder`, as well as the dataset modified from [24]. We hope this repository will provide all necessary resources for facilitating further research and development based on our framework.

References

[1] Daley, A.J., Bloch, I., Kokail, C., Flannigan, S., Pearson, N., Troyer, M., Zoller, P.: Practical quantum advantage in quantum simulation. Nature **607**(7920), 667–676 (2022)

[2] Huang, H.-Y., Broughton, M., Cotler, J., Chen, S., Li, J., Mohseni, M., Neven, H., Babbush, R., Kueng, R., Preskill, J., *et al.*: Quantum advantage in learning from experiments. Science **376**(6598), 1182–1186 (2022)

[3] Renner, R., Wolf, R.: Quantum advantage in cryptography. AIAA Journal **61**(5), 1895–1910 (2023)

[4] Lee, S., Lee, J., Zhai, H., Tong, Y., Dalzell, A.M., Kumar, A., Helms, P., Gray, J., Cui, Z.-H., Liu, W., *et al.*: Evaluating the evidence for exponential quantum advantage in ground-state quantum chemistry. Nature communications **14**(1), 1952 (2023)

[5] Bode, T., Ramesh, K., Stollenwerk, T.: Quantum combinatorial optimization beyond the variational paradigm: Simple schedules for hard problems. Physical Review A **111**(3), 032411 (2025)

[6] Madsen, L.S., Laudenbach, F., Askarani, M.F., Rortais, F., Vincent, T., Bulmer, J.F., Miatto, F.M., Neuhaus, L., Helt, L.G., Collins, M.J., *et al.*: Quantum computational advantage with a programmable photonic processor. Nature **606**(7912), 75–81 (2022)

[7] Cross, A., Javadi-Abhari, A., Alexander, T., De Beaudrap, N., Bishop, L.S., Heidel, S., Ryan, C.A., Sivarajah, P., Smolin, J., Gambetta, J.M., *et al.*: Openqasm 3: A broader and deeper quantum assembly language. ACM Transactions on Quantum Computing **3**(3), 1–50 (2022)

[8] Dupuis, N., Buratti, L., Vishwakarma, S., Forrat, A.V., Kremer, D., Faro, I., Puri,

R., Cruz-Benito, J.: Qiskit code assistant: Training llms for generating quantum computing code. In: IEEE LLM Aided Design Workshop, pp. 1–4 (2024)

[9] Vishwakarma, S., Harkins, F., Golecha, S., Bajpe, V.S., Dupuis, N., Buratti, L., Kremer, D., Faro, I., Puri, R., Cruz-Benito, J.: Qiskit humaneval: An evaluation benchmark for quantum code generative models. In: IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 01, pp. 1169–1176 (2024)

[10] Basit, A., Innan, N., Asif, H., Shao, M., Kashif, M., Marchisio, A., Shafique, M.: Pennylang: Pioneering llm-based quantum code generation with a novel pennylane-centric dataset. arXiv preprint arXiv:2503.02497 (2025)

[11] Castelvecchi, D.: Ibm releases first-ever 1,000-qubit quantum chip. Nature **624**(7991), 238–238 (2023)

[12] Moses, S.A., Baldwin, C.H., Allman, M.S., Ancona, R., Ascarrunz, L., Barnes, C., Bartolotta, J., Bjork, B., Blanchard, P., Bohn, M., *et al.*: A race-track trapped-ion quantum processor. Physical Review X **13**(4), 041052 (2023)

[13] Litteken, A., Fan, Y.-C., Singh, D., Martonosi, M., Chong, F.T.: An updated llvm-based quantum research compiler with further openqasm support. Quantum Science and Technology **5**(3), 034013 (2020)

[14] McKay, D.C., Alexander, T., Bello, L., Biercuk, M.J., Bishop, L., Chen, J., Chow, J.M., Córcoles, A.D., Egger, D., Filipp, S., et al.: Qiskit backend specifications for openqasm and openpulse experiments. arXiv preprint arXiv:1809.03452 (2018)

[15] Khammassi, N., Ashraf, I., Someren, J., Nane, R., Krol, A.M., Rol, M.A., Lao, L., Bertels, K., Almudever, C.G.: Openql: A portable quantum programming framework for quantum accelerators. ACM Journal on Emerging Technologies in Computing Systems (JETC) **18**(1), 1–24 (2021)

[16] Amazon Braket Team: Amazon Braket Examples. https://github.com/amazon-braket/amazon-braket-examples (2024)

[17] Ramalho, N.C.L., Souza, H., Lordello Chaim, M.: Testing and debugging quantum programs: The road to 2030. ACM Transaction on Software Engineering Methodology (2025)

[18] Tang, H., Key, D., Ellis, K.: Worldcoder, a model-based llm agent: Building world models by writing code and interacting with the environment. In: Advances in Neural Information Processing Systems, vol. 37, pp. 70148–70212 (2024)

[19] Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., Ji, H.: Executable code actions elicit better LLM agents. In: Forty-first International Conference on Machine Learning (2024)

[20] microsoft: Building applications with GitHub Copilot agent mode. https://learn.microsoft.com/en-us/training/modules/github-copilot-agent-mode/ (2025)

[21] Pan, H., Mudur, N., Taranto, W., Tikhanovskaya, M., Venugopalan, S., Bahri, Y., Brenner, M.P., Kim, E.-A.: Quantum many-body physics calculations with large language models. Communications Physics **8**(1), 49 (2025)

[22] Zhou, S., Chen, R., An, Z., Zhang, C., Hou, S.-Y.: Application of large language models to quantum state simulation. Science China Physics, Mechanics & Astronomy **68**(4), 240313 (2025)

[23] Jern, L., Uotila, V., Yu, C., Zhao, B.: Fine-tuning large language models on quantum optimization problems for circuit generation. arXiv preprint arXiv:2504.11109 (2025)

[24] Yang, R., Gu, Y., Wang, Z., Liang, Y., Li, T.: Qcircuitnet: A large-scale hierarchical dataset for quantum algorithm design. arXiv preprint arXiv:2410.07961 (2024)

[25] Nakaji, K., Kristensen, L.B., Campos-Gonzalez-Angulo, J.A., Vakili, M.G., Huang, H., Bagherimehrab, M., Gorgulla, C., Wong, F., McCaskey, A., Kim, J.-S., et al.: The generative quantum eigensolver (gqe) and its application for ground state search. arXiv preprint arXiv:2401.09253 (2024)

[26] Liang, Z., Cheng, J., Yang, R., Ren, H., Song, Z., Wu, D., Qian, X., Li, T., Shi, Y.: Unleashing the potential of llms for quantum computing: A study in quantum architecture design. arXiv preprint arXiv:2307.08191 (2023)

[27] Kashani, S.: Quantumllminstruct: A 500k llm instruction-tuning dataset with problem-solution pairs for quantum computing. arXiv preprint arXiv:2412.20956 (2024)

[28] Sinha, P., Jha, S.K., Raj, S.: Circuit partitioning using large language models for quantum compilation and simulations. arXiv preprint arXiv:2505.07711 (2025)

[29] d'Aloisio, G., Fortz, S., Hanna, C., Fortunato, D., Bensoussan, A., Mendiluze Usandizaga, E.n., Sarro, F.: Exploring llm-driven explanations for quantum algorithms. In: ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, pp. 475–481. Association for Computing Machinery, New York, NY, USA (2024)

[30] Zou, Y., Cheng, A.H., Aldossary, A., Bai, J., Leong, S.X., Campos-Gonzalez-Angulo, J.A., Choi, C., Ser, C.T., Tom, G., Wang, A., et al.: El agente: An autonomous agent for quantum chemistry. arXiv preprint arXiv:2505.02484 (2025)

[31] Cao, S., Zhang, Z., Alghadeer, M., Fasciati, S.D., Piscitelli, M., Bakr, M., Leek, P., Aspuru-Guzik, A.: Agents for self-driving laboratories applied to quantum computing (2025). https://arxiv.org/abs/2412.07978

[32] Campbell, C., Chen, H.M., Luk, W., Fan, H.: Enhancing llm-based quantum code generation with multi-agent optimization and quantum error correction. In: ACM/IEEE Design Automation Conference (2025)

[33] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

7 Supplemental Material

7.1 Dynamic-few-shot Coder Example

Figure 5 illustrates an example workflow of QAgent's Dynamic-few-shot Coder for Grover's search problem with $n = 3$. QAgent first retrieves relevant examples and constructs a reasoning-driven prompt to guide OpenQASM code generation. The initial generated code fails due to a syntax error caused by an undefined MCX gate. QAgent subsequently employs CoT-based debugging to pinpoint and resolve the issue. The second iteration successfully yields a syntactically and semantically correct OpenQASM circuit.

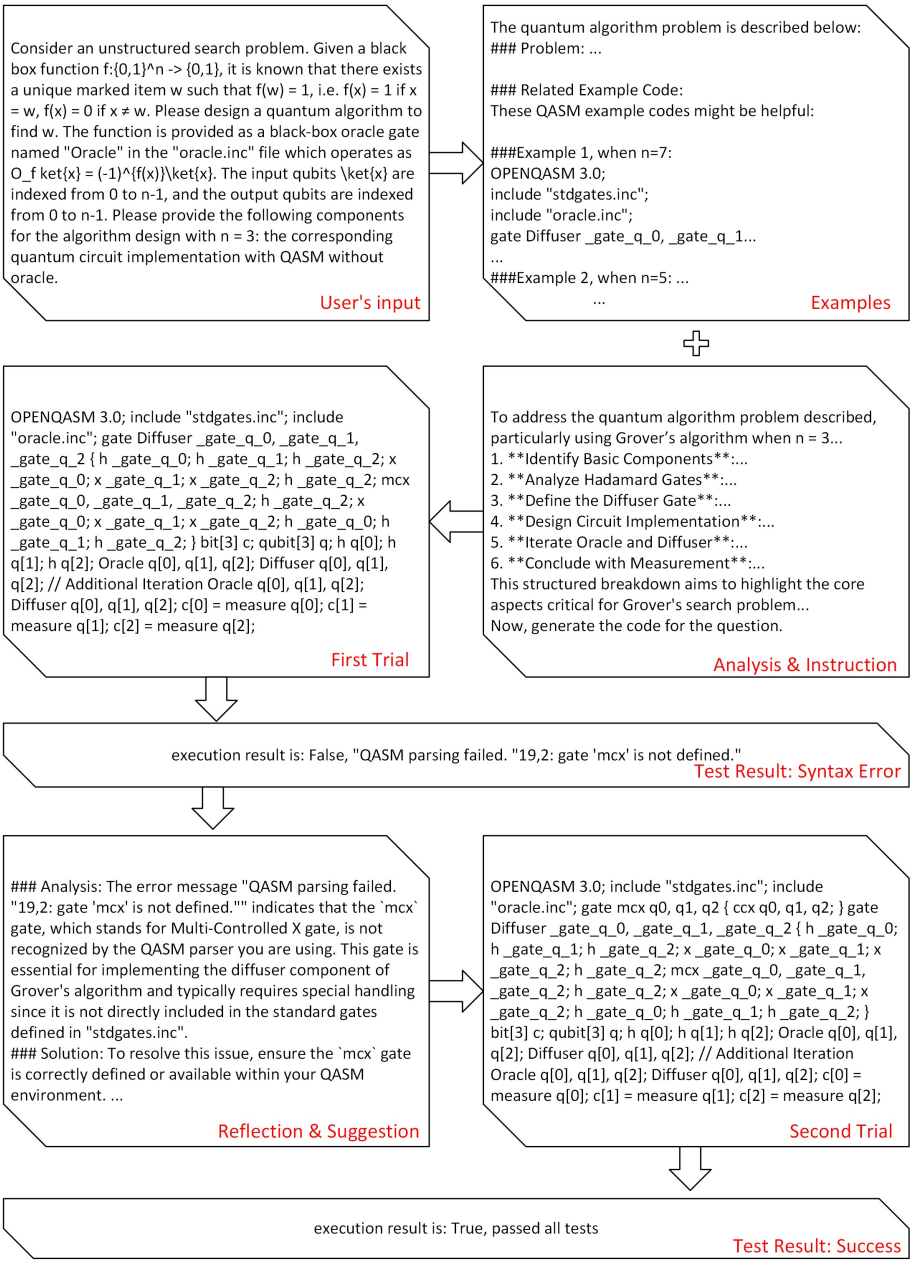


Fig. 5: Example execution trace of the Dynamic-few-shot Coder.

7.2 Tools-augmented Coder Example

Figure 6 presents an example of QAgent's Tools-augmented Coder workflow for Grover's search with $n = 3$. Initially, QAgent formulates a detailed, step-by-step plan and generates Python code using predefined tool functions. The first attempt encounters an error due to an outdated dependency (`qiskit.aqua` module). QAgent then leverages CoT-based debugging to revise the plan according to updated APIs. The subsequent execution successfully generates the expected OpenQASM circuit.

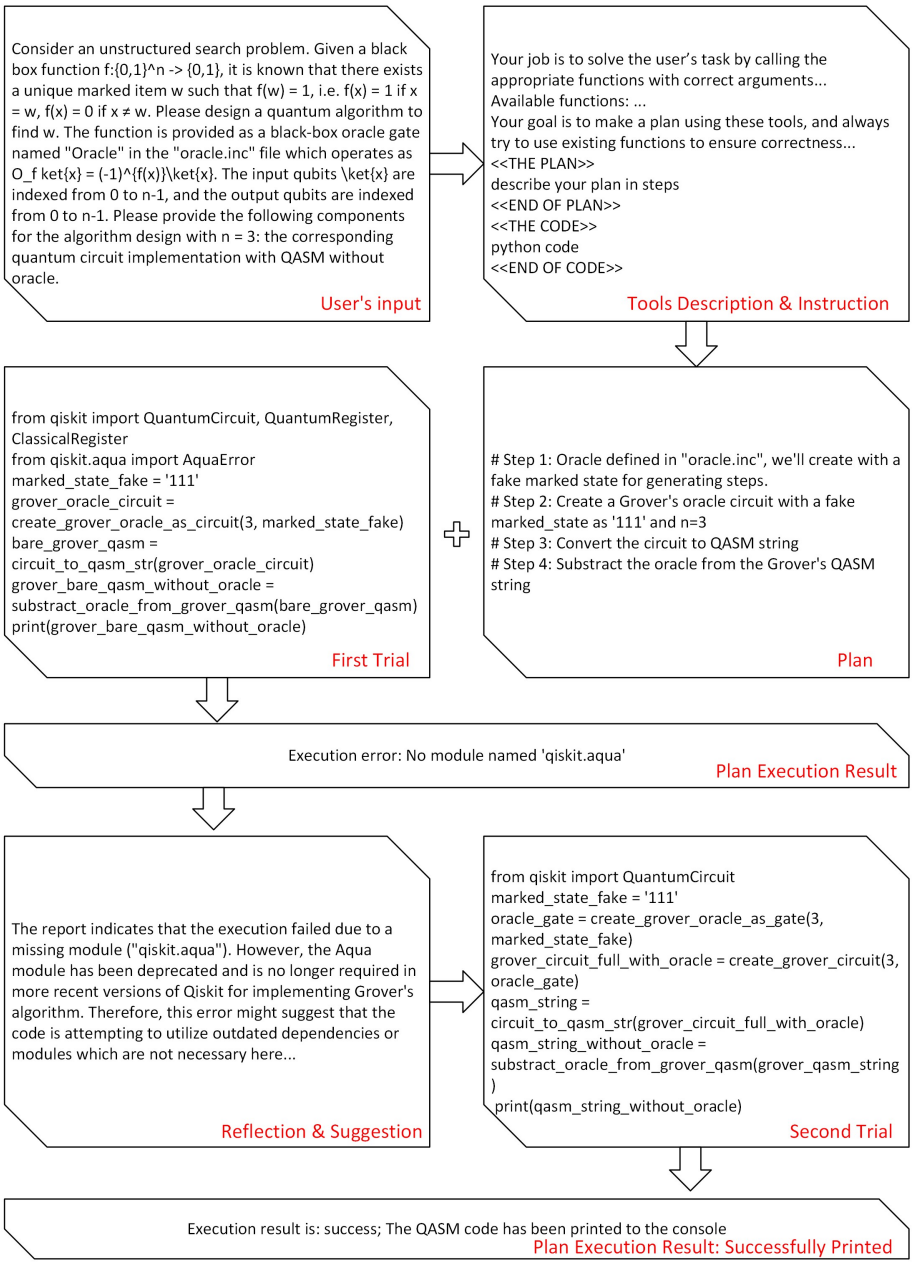


Fig. 6: Example execution trace of the Tools-augmented Coder.

7.3 Typical Error Examples (pe and ws)

Phase Estimation.

When `qubit` $= 4$, a correct OPENQASM program for Phase Estimation is:

```
OPENQASM 3.0;
```

```
include "stdgates.inc";
include "oracle.inc";
gate IQFT _gate_q_0, _gate_q_1, _gate_q_2, _gate_q_3 {
  swap _gate_q_0, _gate_q_3;
  swap _gate_q_1, _gate_q_2;
  h _gate_q_0;
  cp(-pi/2) _gate_q_0, _gate_q_1;
  h _gate_q_1;
  cp(-pi/4) _gate_q_0, _gate_q_2;
  cp(-pi/2) _gate_q_1, _gate_q_2;
  h _gate_q_2;
  cp(-pi/8) _gate_q_0, _gate_q_3;
  cp(-pi/4) _gate_q_1, _gate_q_3;
  cp(-pi/2) _gate_q_2, _gate_q_3;
  h _gate_q_3;}
bit[4] c;
qubit[5] q;
Psi q[4];
h q[0];
h q[1];
h q[2];
h q[3];
CU_0 q[0], q[4];
CU_0 q[1], q[4];
CU_0 q[1], q[4];
CU_0 q[2], q[4];
CU_0 q[2], q[4];
CU_0 q[2], q[4];
CU_0 q[2], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
CU_0 q[3], q[4];
IQFT q[0], q[1], q[2], q[3];
c[0] = measure q[0];
c[1] = measure q[1];
c[2] = measure q[2];
c[3] = measure q[3];
```

Where quantum gate `CU_0` is pre-defined in `oracle.inc`. In phase_estimation the number of `CU_0` applications is *strict*: each control qubit must effect U^{2^k} on the target. Any miscount (missing or extra repetitions) changes the encoded phase and yields wrong estimates. Typical LLM failure modes include:

- **Wrong `IQFT` arity/definition:** the generated `IQFT` gate does not match the register size (e.g., defined with 3 parameters but applied to 4 qubits), so it cannot be used or produces an incorrect transform.
- **Incorrect repetition schedule for `CU_0`:** miscounting the required powers (e.g., using $1, 2, 3, 6$ repeats instead of $1, 2, 4, 8$), which breaks the U^{2^k} scaling.
- **Control/target mix-ups:** swapping arguments to controlled gates, altering the effective unitary.

W-state Preparation.

A correct 4-qubit W state circuit is:

```
OPENQASM 3.0;
include "stdgates.inc";
bit[4] c;
qubit[4] q;
x q[3];
ry(-pi/3) q[2];
cz q[3], q[2];
ry(pi/3) q[2];
ry(-0.9553166181245093) q[1];
cz q[2], q[1];
ry(0.9553166181245093) q[1];
ry(-pi/4) q[0];
cz q[1], q[0];
ry(pi/4) q[0];
cx q[2], q[3];
cx q[1], q[2];
cx q[0], q[1];
```

The single-qubit `ry` angles are not arbitrary constants; they are computed via the two-qubit *F-gate* pattern

```
def F_gate(theta):
    circuit = QuantumCircuit(2)
    circuit.ry(-theta, 1)
    circuit.cz(0, 1)
    circuit.ry(theta, 1)
    return circuit.to_gate().name = "F"
```

and assembled as:

```
theta_i = acos(sqrt(1.0 / (n - i)))
```

with the sequence (conceptually):

```
for i in range(n-1):
    apply F_gate(acos(sqrt(1/(n-i)))) to (n-1-i, n-2-i)
for i in range(n-1):
    cx n-2-i -> n-1-i
```

A common and dominant LLM failure mode in W-state generation is the inability to *capture the underlying formula and generate the precise numeric constants*. Instead, the LLM often outputs approximate angles or random numbers that break the exact amplitude ratios, resulting in a state with significantly reduced fidelity to the ideal W state.

For both cases, seemingly small deviations (one missing repetition, a flipped sign, or a inaccuate parameter) typically lead to qualitatively incorrect states.